Home Automation System (ActorSystem: HomeAutomation)

External Order-Processor System (ActorSystem: OrderProcessorSystem)

Environment

TemperatureEnvironment «Actor / Scheduled»
- currentCelsius: double
+ create(ActorRef): Behavior<TECommand>
- onTick(Tick): Behavior<TECommand>
- onSetTemperature(SetTemperature): Behavior<TECommand>
- onRequestCurrentTemperature(RequestCurrentT): Behavior<TECommand>
+ createReceive(): Receive<TECommand>

WeatherEnvironment «Actor / Scheduled»
- currentCondition: WeatherCondition
+ create(ActorRef): Behavior<WECommand>
- onTick(Tick): Behavior<WECommand>
- onSetWeather(SetWeather): Behavior<WECommand>
- onRequestCurrentWeather(RequestCurrentWeather): Behavior<WECommand>
- pickDifferentCondition(WeatherCondition): WeatherCondition

MqttEnvironmentClient
- coordinator: ActorRef<Command>
- jsonMapper: ObjectMapper
- reconnectScheduler: ScheduledExecutorService
- mqttClient: IMqttClient
- reconnectTask: ScheduledFuture<?>
- shutdown: boolean
+ connect(): void
- attemptConnect(): void
- startReconnectScheduler(): void
- isConnected(): boolean
+ disconnect(): void
- onMessageReceived(String, MqttMessage): void
- handleTemperatureMessage(...): void
- handleWeatherMessage(...): void
- extractTemperatureFromJson(String, String): double
- extractConditionFromJson(String, String): String
- parseJson(String, String): JsonNode

EnvironmentCoordinator «Actor»
- mode: SimulationMode
- currentTemperature: Temperature
- currentWeather: WeatherCondition
- fixedTemperature: double
- fixedWeather: WeatherCondition
- temperatureSensors: Set<ActorRef<TSCommand>>
- weatherSensors: Set<ActorRef<WSCommand>>
+ create(): Behavior<Command>
+ createReceive(): Receive<Command>
- onTemperatureSensorsUpdated(TSUpdated): Behavior<Command>
- onWeatherSensorsUpdated(WSUpdated): Behavior<Command>
- onSetMode(SetMode): Behavior<Command>
- onSetFixedTemperature(SetFixedTemperature): Behavior<Command>
- onSetFixedWeather(SetFixedWeather): Behavior<Command>
- onInternalTemperatureUpdate(InternalTUpdate): Behavior<Command>
- onInternalWeatherUpdate(InternalWUpdate): Behavior<Command>
- onMqttTemperatureUpdate(MqttTUpdate): Behavior<Command>
- onMqttWeatherUpdate(MqttWUpdate): Behavior<Command>
- onGetCurrentState(GetCurrentState): Behavior<Command>
- pushTemperature(double): void
- pushWeather(WeatherCondition): void

SimulationMode «enumeration»
INTERNAL
EXTERNAL_MQTT
FIXED
DISABLED
+ fromString(String): SimulationMode

EnvironmentSnapshot «record»
+ temperature: Temperature
+ weather: WeatherCondition
+ mode: SimulationMode

devices.sensor

TemperatureSensor «Actor»
- airConditions: Set<ActorRef<AirConditionCommand>>
- enabled: boolean
+ create(): Behavior<TSCommand>
+ createReceive(): Receive<TSCommand>
- onTemperatureMeasured(TemperatureMeasured): Behavior<TSCommand>
- onSetEnabled(SetEnabled): Behavior<TSCommand>
- onAirConditionsUpdated(ACUpdated): Behavior<TSCommand>

WeatherSensor «Actor»
- blinds: Set<ActorRef<BlindsCommand>>
- enabled: boolean
+ create(): Behavior<WSCommand>
+ createReceive(): Receive<WSCommand>
- onWeatherMeasured(WeatherMeasured): Behavior<WSCommand>
- onSetEnabled(SetEnabled): Behavior<WSCommand>
- onBlindsUpdated(BlindsUpdated): Behavior<WSCommand>

WeightSensor «Actor / Child»
- identifier: String
- currentTotalKg: double
+ create(String, double): Behavior<WeightSensorCommand>
+ createReceive(): Receive<WeightSensorCommand>
- onWeightChanged(WeightChanged): Behavior<WSCommand>
- onGetWeight(GetWeight): Behavior<WSCommand>
- onPostStop(): Behavior<WSCommand>

SpaceSensor «Actor / Child»
- identifier: String
- currentTotalItems: int
+ create(String, int): Behavior<SpaceSensorCommand>
+ createReceive(): Receive<SpaceSensorCommand>
- onSpaceChanged(SpaceChanged): Behavior<SSCommand>
- onGetSpace(GetSpace): Behavior<SSCommand>
- onPostStop(): Behavior<SSCommand>

shared.model.enviroment

Temperature «record»
+ value: double
+ unit: String
+ celsius(double): Temperature
+ clampToRange(double): double
+ isInRange(double): boolean
+ toString(): String

WeatherCondition «enumeration»
SUNNY
RAINY
CLOUDY
SNOWY
STORMY
+ fromString(String): WeatherCondition

Devices: Actuators

AirCondition «Actor»
- identifier: String
- isCooling: boolean
- isPoweredOn: boolean
+ create(String): Behavior<ACCommand>
+ createReceive(): Receive<ACCommand>
- onTemperatureReceived(EnrichedTemperature): Behavior<ACCommand>
- onPowerAirCondition(PowerAirCondition): Behavior<ACCommand>
- onGetStatus(GetStatus): Behavior<ACCommand>
- onPostStop(): AirCondition

Blinds «Actor»
- identifier: String
- currentWeather: WeatherCondition
- isMoviePlaying: boolean
- areClosed: boolean
+ create(String): Behavior
+ createReceive(): Receive<BlindsCommand>
- onWeatherUpdate(WeatherUpdate): Behavior<BCommand>
- onMovieStatusChanged(MovieStatusChanged): Behavior<BCommand>
- onGetStatus(GetStatus): Behavior<BCommand>
- updateBlindsState(): void
- onPostStop(): Behavior<BCommand>

MediaStation «Actor»
- identifier: String
- blinds: Set<ActorRef<BlindsCommand>>
- isPlaying: boolean
- currentMovie: String
+ create(String): Behavior<MSCommand>
+ createReceive(): Receive<MSCommand>
- onPlayMovie(PlayMovie): Behavior<MSCommand>
- onStopMovie(StopMovie): Behavior<MSCommand>
- onGetStatus(GetStatus): Behavior<MSCommand>
- onBlindsUpdated(BlindsUpdated): Behavior<MSCommand>
- notifyBlinds(boolean): void
- onPostStop(): Behavior<MSCommand>

Fridge «Actor»
- identifier: String
- maxItems: int
- maxWeightKg: double
- inventory: Map<String, Product>
- orderHistory: List<Order>
- grpcClient: OrderServiceClient
- weightSensor: ActorRef<WeightSensorCommand>
- spaceSensor: ActorRef<SpaceSensorCommand>
- pendingItemCount: int
- pendingWeightKg: double
+ create(...): Behavior<FridgeCommand>
- storedItemCount(): int
- storedWeightKg(): double
- committedItemCount(): int
- committedWeightKg(): double
- notifySensors(): void
+ createReceive(): Receive<FridgeCommand>
- onGetProducts(GetProducts): Behavior<FCommand>
- onGetOrderHistory(GetOrderHistory): Behavior<FCommand>
- onGetCapacity(GetCapacity): Behavior<FCommand>
- onConsumeProduct(ConsumeProduct): Behavior<FCommand>
- onOrderProducts(OrderProducts): Behavior<FCommand>
- onOrderCompleted(OrderCompleted): Behavior<FCommand>
- onOrderFailed(OrderFailed): Behavior<FCommand>
- releaseReservation(Order): void
- replaceOrderInHistory(Order): void
- validateAndBuildLineItems(Map): List<OrderLineItem>
- triggerAutoOrder(String): void
- onOrderProcessorCrashed(OrderProcessorCrashed): Behavior<FCommand>
- onPostStop(): Behavior<FCommand>

OrderProcessor «Actor / Per-Session Child»
- grpcClient: OrderServiceClient
- fridge: ActorRef<FridgeCommand>
+ create(...): Behavior<OPCommand>
+ createReceive(): Receive<OPCommand>
- onProcessOrder(ProcessOrder): Behavior<OPCommand>
- onGrpcResponse(GrpcResponse): Behavior<OPCommand>
- onGrpcFailure(GrpcFailure): Behavior<OPCommand>

HTTP / UI Layer

HttpServer
- homePage: String
- environmentRoutes: EnvironmentRoutes
- fridgeRoutes: FridgeRoutes
- mediaRoutes: MediaRoutes
- statusRoutes: StatusRoutes
+ createRoute(): Route
- withCors(Route): Route
- loadHomePage(): String

Routes: (uihandler.routes)

EnvironmentRoutes
- coordinator: ActorRef<ECCommand>
- airConditionActor: ActorRef<ACCommand>
+ routes(): Route
- setFixedTemperature(): Route
- setFixedWeather(): Route
- setSimulationMode(): Route
- setAcPower(): Route

FridgeRoutes
- fridgeActor: ActorRef<FridgeCommand>
- system: ActorSystem<?>
+ routes(): Route
- getProducts(): Route
- getCapacity(): Route
- consumeProduct(): Route
- orderProducts(): Route
- getOrderHistory(): Route
- toOrderDto(Order): OrderDto
- toOrderResponseDto(OrderResponse): OrderResponseDto

MediaRoutes
- mediaStationActor: ActorRef<MSCommand>
- system: ActorSystem<?>
+ routes(): Route
- playMovie(): Route
- stopMovie(): Route
- getStatus(): Route

StatusRoutes
- environmentCoordinator: ActorRef<ECCommand>
- airConditionActor: ActorRef<ACCommand>
- blindsActor: ActorRef<BlindsCommand>
- mediaStationActor: ActorRef<MSCommand>
+ routes(): Route
- getAggregatedStatus(): Route

Domain Model: Order

Product «record»
+ id: String
+ name: String
+ weight: double
+ price: double
+ quantity: int
+ initialQuantity: int
+ withQuantity(int): Product
+ addQuantity(int): Product
+ removeQuantity(int): Product
+ totalWeight(): double
+ equals(Object): boolean
+ hashCode(): int
+ toString(): String

Order «record»
+ orderId: String
+ lineItems: List<OrderLineItem>
+ timestamp: LocalDateTime
+ status: OrderStatus
+ totalPrice: double
+ receiptId: String
+ create(List<OrderLineItem>): Order
+ withStatus(OrderStatus): Order
+ completed(String): Order
+ failed(): Order
+ toString(): String

OrderLineItem «record»
+ productId: String
+ productName: String
+ quantity: int
+ unitPrice: double
+ weightPerUnit: double
+ totalPrice(): double
+ totalWeight(): double

Receipt «record»
+ receiptId: String
+ orderId: String
+ timestamp: LocalDateTime
+ lineItems: List<OrderLineItem>
+ totalPrice: double
+ create(...): Receipt
+ toString(): String

FridgeInventory
+ defaultProducts(): List<Product>

OrderStatus «enumeration»
PENDING
PROCESSING
COMPLETED
FAILED

Exception Hierarchy (shared.model.exceptions)

DomainException «abstract»
- errorCode: ErrorCode
+ getErrorCode(): ErrorCode
+ toString(): String

FridgeException «abstract»

InsufficientSpaceException
InsufficientWeightCapacityException
InvalidOrderException
ProductNotAvailableException
InvalidModeException
InvalidTemperatureException
InvalidWeatherConditionException
MqttConnectionException
InvalidRequestException

ErrorCode «enumeration»
ENV_INVALID_TEMPERATURE
ENV_INVALID_WEATHER_CONDITION
MQTT_CONNECTION_FAILURE
MQTT_MESSAGE_PARSING_FAILURE
FRIDGE_INSUFFICIENT_SPACE
FRIDGE_INSUFFICIENT_CAPACITY
FRIDGE_PRODUCT_NOT_AVAILABLE
FRIDGE_INVALID_ORDER
FRIDGE_INCALID_PRODUCT
FRIDGE_INVALID_LINE_ITEM
INVALID_MODE
INVALID_REQUEST

Exception Handler: (uihandler.exceptions)

GlobalExceptionHandler
+ create(): ExceptionHandler
- buildHandler(): ExceptionHandler
- respond(StatusCode, DomainException): Route

ErrorResponse «record»
+ errorCode: String
+ message: String

DTOs (uihandler.dtos)

StatusDto
ProductDto
ProductsDto
OrderDto
OrderItemDto
OrderHistoryDto
ReceiptDto
OrderRequest
OrderResponseDto
CapacityDto
MediaStatusResponse
SuccessResponse

External Order-Processor System (ActorSystem: OrderProcessorSystem)

OrderProcessorSystem
+ main(String[] args): void

OrderProcessorGuardian «Actor / Guardian»
- serverbinding: ServerBinding
+ create(): Behavior<Command>
- startGrpcServer(ActorRef<ValidationActorCommand>): void
+ createReceive(): Receive<Command>

OrderServiceActorImpl «gRPC Service Impl»
- system: ActorSystem<?>
- validationActor: ActorRef<ValidationActorCommand>
+ processOrder(OrderRequest): CompletionStage<OrderResponse>
- toOrderItemData(OrderRequest): List<OrderItemData>
- toGrpcResponse(ValidationResult): OrderResponse
- buildSuccessResponse(ValidationResult): OrderResponse

ValidationActor «Actor»
- persistenceActor: ActorRef<PACommand>
+ create(ActorRef): Behavior<PACommand>
+ createReceive(): Receive<Command>
- onValidate(ValidateOrder): Behavior<Command>
- findValidationError(List<OrderItemData>): String

PersistenceActor «Actor / EventSourcedBehavior»
- context: ActorContext<Command>
+ create(): Behavior<Command>
+ emptyState(): OrderState
+ commandHandler(): CommandHandler<Command, OrderPersisted, OrderState>
- onPersistOrder(OrderState, PersistOrder): Effect<OrderPersisted, OrderState>
- roundToCents(double): double
+ eventHandler(): EventHandler<OrderState, OrderPersisted>

H2 Database
event_journal /
snapshot

Legende

= Pekko Actor
= External System Actor
= Enumeration
= Record / Value Object / Utility
= Package
= Pekko Receptionist (built-in)
= gRPC Kommunikation (Systemgrenze)
= Receptionist register() / subscribe()